

Best Time to Buy and Sell Stock

Company: JP Morgan

Round: Online Assessment (OA)

Year: 2025

Difficulty: Easy

Topic: Arrays, Greedy

Source: LeetCode 121 – Best Time to Buy and Sell Stock

Problem Description

You are given an integer array `prices`, where `prices[i]` represents the price of a stock on the *i*th day.

You need to choose:

- **One day to buy** the stock.
- **A different day in the future to sell** the stock.

The goal is to maximize the profit.

The profit is calculated as:

profit = selling price - buying price

If no profitable transaction is possible, return 0.

The important condition is that the **buying day must come before the selling day**.

Input Format

- The first line contains a single integer *N* — the number of days for which stock prices are given.
- The second line contains *N* space-separated integers denoting the array prices, where `prices[i]` represents the stock price on the *i*th day.

Output Format

- □ Print a single integer — the maximum profit that can be achieved by buying the stock on one day and selling it on a different future day.
- If no profit can be achieved, print 0.

Constraints

- $1 \leq N \leq 10^5$
- $0 \leq \text{prices}[i] \leq 10^4$

Sample Input / Output

Example 1

Input:

N = 6

`prices` = [7,1,5,3,6,4]

Output:

5

Explanation

The best choice is:

Buy at price = 1

Sell at price = 6

Therefore:

Profit = 6 - 1 = 5

The buying day comes before the selling day, so this transaction is valid.

Example 2

Input:

N = 5

prices = [7,6,4,3,1]

Output:

0

Explanation

The stock price keeps decreasing, so there is no opportunity to make a profit.

Therefore:

Maximum Profit = 0

Approach to Solve This Problem:

The key idea is to keep track of the **lowest stock price seen so far**.

As we move from left to right through the array:

1. Treat the current price as a possible selling price.
2. Keep track of the minimum price seen before the current day.
3. Calculate the profit if we buy at that minimum price and sell today.
4. Keep updating the maximum profit.

For example:

prices = [7,1,5,3,6,4]

Initially:

minPrice = 7
maxProfit = 0
When we see 1:
minPrice = 1
When we see 5:
profit = 5 - 1 = 4
maxProfit = 4
When we see 3:
profit = 3 - 1 = 2
No update to maxProfit.
When we see 6:
profit = 6 - 1 = 5
maxProfit = 5
When we see 4:
profit = 4 - 1 = 3
So the final answer is:
5

Java Solution:

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        int[] prices = new int[n];

        for (int i = 0; i < n; i++) {
            prices[i] = sc.nextInt();
        }

        int minPrice = prices[0];
        int maxProfit = 0;

        for (int i = 1; i < n; i++) {

            // Find the minimum buying price
            if (prices[i] < minPrice) {
                minPrice = prices[i];
            }

            // Calculate profit if we sell today
            int currentProfit = prices[i] - minPrice;
```

```
// Update maximum profit
if (currentProfit > maxProfit) {
    maxProfit = currentProfit;
}
}

System.out.println(maxProfit);
}
}
```

Solution:

1. **Read the input:** main() reads N, then reads N integers into prices, matching the Input Format.
2. **Initialize the minimum price:** Set minPrice to the first element of the array. This represents the lowest buying price found so far.
3. **Initialize maximum profit:** Set maxProfit = 0. If the stock price only decreases, no profitable transaction is possible, so the answer remains 0.
4. **Traverse the array:** Starting from the second element, check each stock price one by one. If the current price is smaller than minPrice, update minPrice because this gives us a better buying price for future days.
5. **Calculate the possible profit:** For every current price, calculate currentProfit = prices[i] - minPrice. This represents the profit we would make by buying at the lowest price seen so far and selling on the current day.
6. **Update maximum profit:** If currentProfit is greater than maxProfit, update maxProfit. Since minPrice always comes from an earlier day, the buying day is guaranteed to be before the selling day.
7. **Print the result:** After traversing the complete array, print maxProfit. If no profitable transaction exists, it remains 0, matching the Output Format.

Complexity Analysis:

- ☐ Time Complexity: $O(N)$ — the array is traversed only once.
- ☐ Space Complexity: $O(1)$ extra space — only minPrice, maxProfit, and currentProfit are used.

